

Modern GPU Rendering Techniques in High-Graphic Video Games: An Architectural and Algorithmic Analysis

Swastik Pandey

National Institute of Technology, Jalandhar
swastikp.it.25@nitj.ac.in

Abstract

Modern high-fidelity video games represent one of the most computationally demanding real-time applications in existence. The visual quality achieved in titles such as *Cyberpunk 2077*, *Alan Wake 2*, and *God of War* is not the result of any single breakthrough technology, but rather the coordinated orchestration of dozens of advanced GPU rendering techniques operating in parallel within a single frame budget of 16 to 33 milliseconds. This paper provides a comprehensive, mathematically grounded survey of the core architectural and algorithmic foundations underlying modern real-time rendering. Beginning with GPU hardware architecture, the discussion progresses through the programmable graphics pipeline, rasterization mathematics, physically based rendering, hardware-accelerated ray tracing, path tracing, next-generation geometry virtualization, and finally AI-driven image reconstruction and frame generation. The goal is to equip the reader with a thorough academic understanding of not only *what* these techniques accomplish, but *how* they accomplish it at the hardware and mathematical level.

1. Introduction: GPU Architecture and the Programmable Pipeline

1.1 Historical Context and the GPU's Emergence

To understand why the GPU was invented, one must first understand the computational demands of real-time 3D graphics. In the early 1990s, all graphics calculations were performed on the CPU. Rendering a single textured, lit 3D polygon required floating-point multiply-accumulate operations, matrix transformations, division for perspective correction, texture coordinate lookups, and color blending — all performed sequentially by a processor never designed for such workloads. As game resolution, polygon counts, and scene complexity grew, it became clear that the CPU could not scale to meet graphics demands.

The introduction of dedicated graphics accelerators, culminating in NVIDIA's landmark **GeForce 256** in 1999 (the first card marketed as a "GPU"), moved

transform and lighting calculations off the CPU. However, this hardware was entirely fixed-function: it could only perform specific, hard-coded operations. The revolutionary shift came in 2001 with the **GeForce 3**, which introduced the first programmable vertex and pixel shaders, enabling developers to write arbitrary mathematical programs to execute on GPU hardware. This programmability is the foundation upon which all modern rendering techniques are built.

1.2 Architectural Divergence: CPU vs. GPU

The fundamental divergence in computer architecture between the **Central Processing Unit (CPU)** and the **Graphics Processing Unit (GPU)** dictates the rendering algorithms that are feasible in real-time. To understand this divergence, one must consider the physics of computation: both processors are bound by the same silicon manufacturing processes, yet they make radically different trade-offs in allocating their transistor budgets.

The CPU is a **latency-oriented processor**. Its dominant use case is general-purpose sequential computation — running operating systems, compilers, game logic, and physics simulations. These workloads involve frequent conditional branching, pointer-chasing through memory, and data-dependent control flow. To minimize the time-to-result for a single instruction stream, the CPU dedicates substantial silicon area to:

- **Large, Multi-Level Cache Hierarchies (L1, L2, L3):** Aggressively cache frequently accessed data to reduce expensive DRAM latency from ~100 nanoseconds to as little as ~1 nanosecond.
- **Branch Prediction Hardware:** Speculatively execute instructions along the most probable branch of a conditional, discarding speculative results if the prediction proves incorrect.
- **Out-of-Order Execution Engines:** Analyze a window of upcoming instructions, identify those with resolved dependencies, and execute them ahead of older, stalled instructions to maximize unit utilization.
- **Superscalar Dispatch:** Issue multiple independent instructions per clock cycle to multiple execution units simultaneously.

These features allow a single CPU core to execute sequential code at extraordinary speeds. However, they consume enormous silicon area and power, leaving room for only a small number of physical cores (typically 8 to 32 in modern high-end desktop processors).

The GPU takes the diametrically opposite approach. Graphics workloads are characterized by extreme data parallelism: computing the shading of one pixel is entirely independent of computing the shading of any other pixel. This regular, predictable parallelism is ideally suited for a **throughput-oriented** design philosophy. The GPU dedicates the overwhelming majority of its silicon to raw **Arithmetic Logic Units (ALUs)**, with minimal cache and comparatively simple control logic.

Rather than hiding memory latency through caching, the GPU hides it through **latency hiding via massive thread-level parallelism**. The key insight is that if you maintain a sufficiently large number of threads in-flight simultaneously, whenever any group of threads stalls waiting for a memory fetch, another group is immediately available to execute, keeping the ALUs occupied continuously.

When a group of threads — called a **Warp** (NVIDIA, 32 threads) or a **Wavefront** (AMD, 32 or 64 threads) — stalls on a memory access, the hardware scheduler performs a zero-cost context switch to another warp that is ready to execute. This zero-cost switching is possible because the complete register state of all resident warps is maintained simultaneously in a massive on-chip register file; unlike the CPU, there is no need to save state to a cache or memory. Modern GPUs can maintain thousands of warps in-flight simultaneously across their execution units.

Architectural Feature	Central Processing Unit (CPU)	Graphics Processing Unit (GPU)
Primary Design Goal	Minimize single-thread latency	Maximize aggregate throughput
Silicon Budget Allocation	Cache, control logic, OOO engines	Arithmetic Logic Units (ALUs)
Physical Core Count	8-32 large, complex cores	Thousands of small, simple shader processors
Memory Latency Strategy	Large L1/L2/L3 caches, prefetching	Latency hiding via massive in-flight thread count
Context Switch Cost	High (must save registers to cache)	Near-zero (register file holds all warp states)
Thread Execution Model	Independent, divergent execution	SIMT (32/64 threads execute identical instruction simultaneously)
Suited For	Branchy, sequential, pointer-chasing code	Regular, parallel, data-independent compute kernels

Major Hardware Limitation - SIMT Divergence: The SIMT model requires all threads in a warp to execute the *same* instruction at the same time. If threads encounter a conditional branch (**if/else**) where some evaluate **true** and others evaluate **false**, the hardware cannot execute both paths in parallel. It must serialize execution: first executing the **if** branch with the **else** threads masked off (inactive), then executing the **else** branch with the **if** threads masked off. This **divergent branching** cuts effective ALU utilization in half (or worse), and is one of the most critical micro-architectural pitfalls in GPU shader programming.

1.3 GPU Memory Subsystem Architecture

The GPU memory subsystem is a critical architectural component distinct from the CPU's memory model. Modern discrete GPUs contain dedicated **Video RAM (VRAM)** - typically GDDR6X or HBM3 - connected via an extremely wide memory bus (256-bit to 384-bit, or in the case of HBM, over 1024-bit) to provide memory bandwidth often exceeding 1 TB/s. This bandwidth is essential because the GPU must simultaneously feed texture data, vertex buffers, shader constants, and render target reads/writes to thousands of concurrent shader invocations.

The on-chip memory hierarchy of a GPU consists of: * **L1 Cache / Shared Memory**: A small, extremely fast memory (32-128 KB per compute unit) that can be configured either as a hardware-managed L1 cache or as programmer-managed **shared memory** (also called Local Data Share, LDS, on AMD). Shared memory allows threads within the same workgroup to communicate and share intermediate results. * **L2 Cache**: A larger, unified cache shared across all compute units (typically 4-32 MB on modern GPUs), primarily used to coalesce texture accesses and reduce VRAM bandwidth consumption. * **Texture Cache**: Specialized hardware optimized for 2D spatial locality, accelerating the bilinear and trilinear filtering operations performed during texture sampling. * **VRAM (Global Memory)**: The largest but slowest memory tier, holding the bulk of scene data - vertex buffers, index buffers, texture atlases, render targets, and shader resource buffers.

Memory access patterns that exhibit high **spatial locality** (accessing nearby addresses simultaneously across threads) allow the memory controller to coalesce multiple small transactions into a single wide VRAM access, dramatically improving effective bandwidth. Scattered or non-coalesced memory access patterns result in multiple separate VRAM transactions and represent a major cause of GPU performance degradation in poorly optimized shaders.

1.4 The Logical Graphics Pipeline and Memory Data Flow

The traditional graphics pipeline defines a sequence of stages - some programmable, some fixed-function - through which raw geometric data flows to produce a final 2D image in the framebuffer. Understanding this pipeline in depth is essential before appreciating how modern extensions and replacements improve upon it.

1.4.1 Input Assembly (IA) The pipeline begins when the CPU issues a Draw Call through the graphics API (DirectX 12, Vulkan, Metal). The **Input Assembler** stage fetches vertex data from vertex buffers in VRAM and, if indexed rendering is used, fetches index data from an index buffer. It assembles the raw stream of vertex attributes - positions, normals, tangents, texture coordinates, and vertex colors - into discrete primitives (triangles, lines, or points) ready for the vertex shader.

The key optimization here is indexed rendering: rather than duplicating vertex data for every triangle that shares a vertex, each unique vertex is stored once in the vertex buffer, and a smaller index buffer simply references those vertices by integer index. This dramatically reduces VRAM consumption and improves vertex cache hit rates.

1.4.2 Vertex Shader (VS) The **Vertex Shader** is the first programmable stage of the pipeline. It executes once per vertex and cannot create or destroy vertices - it can only read input vertex attributes and write a transformed output. The primary mathematical responsibility of the vertex shader is the **Model-View-Projection (MVP) transformation**, which converts vertex positions through three successive coordinate space transformations:

1. **Model Transform (M):** Converts from the mesh's local object space to world space, encoding translation, rotation, and scale via a 4×4 homogeneous transformation matrix.
2. **View Transform (V):** Converts from world space to camera (view) space, repositioning the world so the camera sits at the origin looking down the negative Z axis.
3. **Projection Transform (P):** Converts from view space to clip space, applying either perspective division (for realistic perspective) or orthographic projection. Perspective projection encodes depth non-linearly into the W component for subsequent hardware division.

The combined operation for a vertex position v is:

$$v_{clip} = P \cdot V \cdot M \cdot v_{local}$$

Beyond MVP transformation, vertex shaders are frequently used for skeletal animation skinning (deforming geometry by blending multiple bone matrices weighted per vertex), wind simulation (displacing vegetation vertices using sinusoidal functions), and vertex morphing for facial animation blend shapes.

1.4.3 Tessellation Stages The tessellation pipeline is an optional three-stage expansion between the vertex shader and the rasterizer, enabling the GPU to subdivide coarse geometric patches into dense meshes of triangles at runtime. It consists of:

1. **Hull Shader (HS):** Executes once per control point of a geometric patch, and once per patch to output tessellation factors - floating-point values that instruct the fixed-function tessellator on how many times to subdivide each edge and the interior of the patch.
2. **Fixed-Function Tessellator:** Hardware that takes the tessellation factors from the hull shader and generates the actual subdivision topology - the new vertices and connectivity information as a regular grid or triangular pattern.

3. **Domain Shader (DS):** Executes once per generated vertex, evaluating the final world position of that vertex by interpolating control points using patch basis functions (e.g., Bezier curves). Critically, the domain shader can sample a **displacement map** texture to push generated vertices along the surface normal, adding geometric detail that matches high-resolution textures without requiring artists to model every surface detail manually.

Tessellation is widely used for terrain rendering (where distant terrain patches receive low tessellation factors and close patches receive high factors) and character skin rendering where smooth, rounded silhouettes are required.

1.4.4 Geometry Shader (GS) The **Geometry Shader** operates on complete, assembled primitives (triangles, lines, points). Unlike the vertex shader, it can emit zero or more output primitives, making it capable of both generating new geometry and discarding existing geometry. Common uses include rendering to all six faces of a cubemap simultaneously (instancing to six render target layers in a single pass), expanding point sprites to screen-aligned quads, and cascaded shadow map rendering.

However, the Geometry Shader has a notoriously poor fit with wide SIMD GPU hardware. Because different triangles may generate different numbers of output primitives, the output size is non-uniform, creating variable-length work per thread that is fundamentally at odds with the GPU's fixed-width SIMT execution model. Most modern rendering pipelines minimize or eliminate its use, replacing its functionality with compute shaders or the Mesh Shader pipeline.

1.4.5 Rasterization The **Rasterizer** is a fixed-function hardware block that bridges the geometric world and the pixel world. It performs three primary functions:

1. **Perspective Division:** Divides the clip-space X, Y, Z coordinates by the W component to produce Normalized Device Coordinates (NDC), mapping geometry into the canonical $[-1, +1]^3$ volume. This division implements the perspective foreshortening effect that makes distant objects appear smaller.
2. **Viewport Transform:** Scales and translates NDC coordinates into window/screen coordinates, mapping the $[-1, +1]$ range to the actual pixel dimensions of the render target.
3. **Triangle Traversal and Coverage Testing:** For each triangle, the rasterizer determines which pixels (and which sub-pixel samples in MSAA rendering) are covered by the triangle's area, generating a fragment for each covered sample. It computes barycentric coordinates for attribute interpolation across the triangle face.

The rasterizer also performs **early depth testing (Early-Z)** if the pixel shader does not modify depth. By testing the depth of incoming fragments against the existing depth buffer before invoking the pixel shader, the GPU can discard

fragments that are occluded by previously rendered geometry, avoiding wasteful execution of expensive pixel shader programs.

1.4.6 Pixel / Fragment Shader The **Pixel Shader** (Direct3D terminology) or **Fragment Shader** (OpenGL/Vulkan terminology) executes once per fragment generated by the rasterizer. This is invariably the most computationally demanding stage of the traditional graphics pipeline, as it performs texture sampling (which may involve filtering across up to 16 texels for trilinear mipmapped samples), BRDF evaluation, shadow map comparisons, screen-space effect calculations, and complex material blending.

The pixel shader receives the interpolated vertex attributes (computed from barycentric coordinates by the rasterizer) and outputs one or more color values. In a **deferred rendering** pipeline, the pixel shader during the geometry pass writes to multiple render targets simultaneously (an operation called **Multiple Render Targets, or MRT**), populating a **G-Buffer** of intermediate data (albedo, normals, metallic/roughness, depth) that is subsequently consumed by a separate lighting pass.

1.4.7 Output Merger and Render Output Units (ROPs) The final fixed-function stage performs per-fragment tests and final framebuffer composition:

- **Depth Test:** Compares the fragment's depth value against the current value in the depth buffer. If the fragment is behind existing geometry, it is discarded. If in front, it writes its depth to the depth buffer.
- **Stencil Test:** Compares the stencil buffer against a reference value with a configurable comparison function, allowing the depth buffer to be masked or selectively written.
- **Alpha Blending:** Combines the pixel shader's output color with the existing framebuffer color using a configurable blend equation, enabling transparency effects.

This stage is implemented in dedicated **Render Output Units (ROPs)**, specialized hardware that provides the atomic read-modify-write capability necessary for correct blending and depth updates. The number of ROPs in a GPU directly limits its pixel fill rate (measured in pixels per second).

2. Mathematical Foundations of Real-Time Rasterization

2.1 The Rasterization Algorithm: Edge Equations and Coverage Testing

Rasterization maps the continuous domain of 3D triangles onto the discrete domain of a 2D pixel grid. The fundamental question it must answer is: **for each pixel, which (if any) triangle covers it?** The hardware solves this using **edge equations**, derived from the signed area of a triangle.

For a directed edge from vertex $V_0 = (x_0, y_0)$ to $V_1 = (x_1, y_1)$, the edge equation evaluates the signed distance of an arbitrary point (x, y) from the edge's line:

$$E_{01}(x, y) = (x - x_0)(y_1 - y_0) - (y - y_0)(x_1 - x_0)$$

A pixel at (x, y) is considered inside the triangle $V_0V_1V_2$ if and only if all three edge equations are simultaneously non-negative (for a counter-clockwise winding order). This test can be evaluated in parallel for every pixel in the triangle's bounding box. The GPU rasterizer computes these edge equations using fixed-point arithmetic on a tile-based basis, processing 8×8 or 16×16 pixel tiles simultaneously.

2.1.1 Barycentric Coordinates and Perspective-Correct Interpolation

Once a pixel is determined to lie within a triangle, its vertex attributes (normals, texture coordinates, colors) must be interpolated across the triangle face. This is accomplished via **barycentric coordinates** $(\lambda_0, \lambda_1, \lambda_2)$, defined such that:

$$\lambda_0 + \lambda_1 + \lambda_2 = 1, \quad \lambda_i \geq 0$$

Each barycentric coordinate is proportional to the ratio of the sub-triangle area formed with the opposite vertex to the total triangle area:

$$\lambda_0 = \frac{E_{12}(x, y)}{E_{12}(x_0, y_0)}, \quad \lambda_1 = \frac{E_{20}(x, y)}{E_{20}(x_1, y_1)}, \quad \lambda_2 = \frac{E_{01}(x, y)}{E_{01}(x_2, y_2)}$$

A naive linear interpolation of attributes using barycentric coordinates produces incorrect results after perspective projection, because the projection is a non-linear transformation. To achieve correct **perspective-correct interpolation**, attributes must be interpolated in view space (before projection). The GPU accomplishes this by dividing the attribute by the vertex W coordinate before rasterization, interpolating the divided attribute linearly, and then multiplying by the interpolated $1/W$ to reconstruct the perspective-correct value:

$$\text{attr}_{interp} = \frac{\lambda_0 \frac{\text{attr}_0}{w_0} + \lambda_1 \frac{\text{attr}_1}{w_1} + \lambda_2 \frac{\text{attr}_2}{w_2}}{\lambda_0 \frac{1}{w_0} + \lambda_1 \frac{1}{w_1} + \lambda_2 \frac{1}{w_2}}$$

This perspective correction is critical for texture mapping - without it, textures appear to swim and distort incorrectly as surfaces recede from the camera.

2.2 The Rendering Equation: The Mathematical Basis of Light Transport

The theoretical foundation of all light simulation in computer graphics is the **Rendering Equation**, first formalized by James T. Kajiya in 1986. It expresses

the equilibrium distribution of radiance (light energy per unit area per unit solid angle) across a scene:

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega^+} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i) d\omega_i$$

Where each term carries precise physical meaning:

Symbol	Physical Meaning	Units
$L_o(p, \omega_o)$	Outgoing radiance from point p in direction ω_o	$\text{W} \cdot \text{sr}^{-1} \cdot \text{m}^{-2}$
$L_e(p, \omega_o)$	Emitted radiance (non-zero for light sources only)	$\text{W} \cdot \text{sr}^{-1} \cdot \text{m}^{-2}$
$f_r(p, \omega_i, \omega_o)$	BRDF: ratio of outgoing to incoming radiance	sr^{-1}
$L_i(p, \omega_i)$	Incoming radiance from direction ω_i	$\text{W} \cdot \text{sr}^{-1} \cdot \text{m}^{-2}$
$(n \cdot \omega_i)$	Lambert's cosine law (projected solid angle factor)	Dimensionless
$d\omega_i$	Differential solid angle over the hemisphere Ω^+	sr

The integral ranges over the entire upper hemisphere Ω^+ of incoming light directions. Evaluating this integral exactly requires knowledge of $L_i(p, \omega_i)$ for every possible incoming direction - but L_i itself depends on the outgoing radiance of whatever surface the incoming ray originates from, creating a recursive definition. This is the fundamental computational challenge of light transport: the integral is self-referential and cannot be solved analytically for complex scenes.

Real-time rendering's entire history is a story of finding efficient, approximate solutions to this integral under the constraint of millisecond frame budgets.

2.3 Physically Based Rendering (PBR) and the Cook-Torrance BRDF

Physically Based Rendering (PBR) is a standardized rendering paradigm that enforces two foundational physical laws in material shading:

1. **Energy Conservation:** A surface cannot reflect more light than it receives. The integral of the BRDF over the hemisphere, weighted by the cosine factor, must be ≤ 1 for all incident directions.

2. Helmholtz Reciprocity: The BRDF is symmetric: $f_r(p, \omega_i, \omega_o) = f_r(p, \omega_o, \omega_i)$. Swapping the incoming and outgoing directions yields the same BRDF value.

In practice, a PBR material is represented by a small set of artist-authored parameters: **Albedo** (base color), **Metallic** (whether the surface is a metal or dielectric), and **Roughness** (microscopic surface irregularity). The BRDF is split into a diffuse lobe and a specular lobe:

$$f_r = f_{diffuse} + f_{specular}$$

The diffuse term for dielectrics uses the **Lambertian model**: $f_{diffuse} = \frac{c_{albedo}}{\pi}$, where the π denominator normalizes it to conserve energy. Metallic surfaces have no diffuse component (absorbed light is not re-emitted at a different wavelength).

The specular term universally uses the **Cook-Torrance Microfacet BRDF**, which models a surface as a collection of microscopic, perfectly flat mirror facets oriented randomly according to a statistical distribution:

$$f_{specular}(p, \omega_i, \omega_o) = \frac{D(h) F(\omega_o, h) G(\omega_i, \omega_o, h)}{4 (n \cdot \omega_i)(n \cdot \omega_o)}$$

Where $h = \frac{\omega_i + \omega_o}{|\omega_i + \omega_o|}$ is the **half-vector** bisecting the incoming and outgoing directions.

2.3.1 Normal Distribution Function (NDF): Trowbridge-Reitz GGX

The **Normal Distribution Function** $D(h)$ describes what fraction of microfacets are oriented precisely in the half-vector direction (and thus contribute to specular reflection). The **GGX (Trowbridge-Reitz)** NDF has become the industry standard due to its physically motivated long-tailed distribution, which produces a realistic bright specular highlight with a gradual falloff:

$$D_{GGX}(h) = \frac{\alpha^2}{\pi ((n \cdot h)^2(\alpha^2 - 1) + 1)^2}$$

Here $\alpha = \text{roughness}^2$ is the squared roughness parameter. A roughness of 0 produces a perfect mirror (all microfacets aligned), while a roughness of 1 produces a completely diffuse-looking specular response (microfacets oriented randomly). The squaring of roughness is an artistic remapping that linearizes the perceptual response of the roughness slider.

2.3.2 Fresnel Equation: Schlick's Approximation

The **Fresnel equation** $F(\omega_o, h)$ governs the ratio of reflected to refracted light at a surface boundary as a function of the viewing angle. At grazing angles (when ω_o is nearly perpendicular to n), all surfaces - including matte dielectrics like wood or plastic - become

highly specular and mirror-like. This is the well-known physical phenomenon observed when looking at a lake at a shallow angle versus looking straight down into it.

The exact Fresnel equations are complex and wavelength-dependent. In real-time rendering, **Schlick's approximation** provides an excellent closed-form estimate:

$$F_{Schlick}(\omega_o, h) = F_0 + (1 - F_0)(1 - (\omega_o \cdot h))^5$$

Where F_0 is the **characteristic specular reflectance at normal incidence** (when the viewing angle is 0 degrees). For dielectric materials (plastic, wood, fabric), F_0 is a small grayscale value (typically 0.02-0.08, corresponding to 2-8% reflectance). For metallic conductors (gold, copper, iron), F_0 is a colored RGB value (often 60-100% reflectance), which is why metals have a tinted specular highlight that matches their surface color.

2.3.3 Geometry Function: Schlick-GGX Smith Approximation The **Geometry Function** $G(\omega_i, \omega_o, h)$ accounts for the self-shadowing and self-masking of microfacets: tall microfacets can block the incoming light from reaching some shorter microfacets (shadowing), and can block the reflected light from reaching the viewer (masking). Without this term, the BRDF would overestimate reflectance for rough surfaces at grazing angles, violating energy conservation.

The **Schlick-GGX approximation** of the Smith shadowing-masking function separates the function into independent shadowing and masking terms:

$$G_{Smith}(\omega_i, \omega_o) = G_{Schlick}(\omega_i) \cdot G_{Schlick}(\omega_o)$$

$$G_{Schlick}(\omega) = \frac{(n \cdot \omega)}{(n \cdot \omega)(1 - k) + k}, \quad k = \frac{(\alpha + 1)^2}{8}$$

2.4 Deferred and Forward Rendering Architectures

Modern game engines must choose between two primary rendering pipeline architectures, each with different trade-offs for scene complexity and light count.

2.4.1 Forward Rendering In **Forward Rendering**, each object is drawn once, and every light affecting it is evaluated inside the single pixel shader pass. While conceptually simple, this approach scales poorly: a scene with N objects and M lights requires up to $N \times M$ shader invocations. Furthermore, geometry hidden behind other objects still incurs the cost of pixel shader execution before depth testing eliminates it. Despite these drawbacks, forward rendering remains relevant for transparent objects and mobile platforms.

2.4.2 Deferred Rendering **Deferred Rendering** splits rendering into two distinct passes:

1. **Geometry Pass (G-Pass):** All opaque geometry is rendered, but the pixel shader only outputs surface properties (albedo, normals, roughness, metallic, depth) into a set of full-resolution render targets collectively called the **G-Buffer**. No lighting is computed in this pass. Because depth testing correctly discards occluded fragments before expensive G-buffer writes, no pixel shader computation is wasted on invisible geometry.
2. **Lighting Pass:** For each light in the scene, a screen-aligned shape (quad for directional lights, sphere mesh for point lights) is rendered. The pixel shader for each light reads the G-Buffer textures at the current screen position and evaluates the BRDF once per light. Lighting cost is thus proportional to the screen-area contribution of each light, not the number of objects in the scene.

The major limitation of deferred rendering is VRAM bandwidth: the G-Buffer may contain 4-6 full-resolution floating-point render targets (up to 48 bytes per pixel at 4K), requiring substantial bandwidth for both writing during the geometry pass and reading during the lighting pass.

2.5 Shadow Mapping and Advanced Shadow Techniques

2.5.1 Shadow Maps: Foundational Mechanism **Shadow Mapping**, introduced by Lance Williams in 1978, remains the dominant real-time shadowing technique. The algorithm renders the scene from the perspective of the light source, recording only the depth of the closest visible geometry into a texture called the **Shadow Map**. During the main camera render, each pixel’s world-space position is transformed into the light’s clip space, and its distance is compared to the stored shadow map depth. If the pixel’s light-space depth exceeds the shadow map depth by more than a small **depth bias** (used to prevent self-shadowing artifacts called “shadow acne”), it is declared in shadow.

Mathematically, for a pixel at world position p_{world} , its light-space position is:

$$p_{light} = P_{light} \cdot V_{light} \cdot p_{world}$$

The shadow test is then:

$$\text{shadow} = \begin{cases} 0 & \text{if } p_{light}.z - \text{bias} \leq \text{ShadowMap}(p_{light}.xy) \\ 1 & \text{otherwise} \end{cases}$$

2.5.2 Cascaded Shadow Maps (CSM) A single shadow map cannot simultaneously provide high resolution for close objects and cover large outdoor environments. **Cascaded Shadow Maps (CSM)** solve this by partitioning the camera’s view frustum into multiple depth slices (cascades) and rendering

a separate shadow map for each cascade. Objects close to the camera fall into Cascade 0 (small, high-resolution shadow map area), and objects progressively farther away fall into higher cascades covering larger areas with lower effective resolution.

Cascade split distances are typically determined using a logarithmic or practical split scheme. The **Practical Split Scheme** linearly blends uniform and logarithmic splits:

$$C_i = \lambda \cdot C_{log,i} + (1 - \lambda) \cdot C_{uniform,i}$$

where λ balances depth coverage against shadow map texel density near the camera.

2.5.3 Percentage-Closer Soft Shadows (PCSS) Standard shadow mapping produces hard-edged shadows, which are physically inaccurate for area lights. Real-world shadows transition from a sharp inner region (**umbra**) to a soft outer region (**penumbra**) proportional to the angular size of the light source. **Percentage-Closer Soft Shadows (PCSS)** physically model this:

1. **Blocker Search:** Sample the shadow map in a region around the current point to find the average depth of geometry blocking the light.
2. **Penumbra Width Estimation:** Calculate the penumbra radius $w_{penumbra}$ based on the receiver depth d_r , blocker depth d_b , and light radius w_{light} :

$$w_{penumbra} = w_{light} \cdot \frac{d_r - d_b}{d_b}$$

3. **Percentage-Closer Filtering (PCF):** Sample the shadow map multiple times within the computed penumbra radius, averaging the binary shadow comparisons to produce a smooth shadow factor.

2.6 Global Illumination Approximations: Ambient Occlusion

2.6.1 Screen Space Ambient Occlusion (SSAO) SSAO, introduced by Crytek in Crysis (2007), approximates the ambient occlusion integral - the fraction of the hemisphere of incoming ambient light that is unoccluded - using only information available in the screen-space depth buffer.

For each pixel, a random set of sample points is generated in a hemisphere oriented along the surface normal. Each sample point is projected into screen space, and its depth is looked up in the depth buffer. If the sample lies below the recorded surface (inside geometry), it contributes to occluding the ambient light. The average occlusion across all samples gives the per-pixel ambient occlusion factor:

$$\text{AO}(p) = 1 - \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{SampleDepth}_i < \text{RecordedDepth}_i]$$

The fundamental limitation of SSAO is that its hemisphere samples are distributed in screen space rather than true 3D world space, leading to false occlusion at depth discontinuities and an inability to detect occluders outside the screen boundaries.

2.6.2 Horizon-Based Ambient Occlusion (HBAO) HBAO, developed by NVIDIA, represents a physically motivated improvement over SSAO. Instead of sampling random 3D hemisphere points, HBAO samples the depth buffer along multiple radial directions around the pixel, finding the maximum elevation angle (horizon angle) of the surrounding geometry in each direction.

The ambient occlusion is computed as the fractional area of the hemisphere blocked by geometry below the horizon angles. This formulation incorporates true surface normal information, computing occlusion relative to the actual surface orientation rather than an assumed upward hemisphere, resulting in more physically plausible occlusion in tight crevices and around curved surfaces.

3. Hardware-Accelerated Ray Tracing and Path Tracing

3.1 The Ray Tracing Paradigm: Image-Order vs. Object-Order Rendering

The fundamental difference between ray tracing and rasterization is the order of iteration. Rasterization is **object-order**: it iterates over geometric primitives and determines which pixels each primitive covers. Ray tracing is **image-order**: it iterates over pixels (or samples), fires rays into the scene, and determines what geometry each ray hits.

This inversion of loop order gives ray tracing an immediate advantage: it has natural access to global spatial information. A ray fired from a pixel can directly query any geometry in the scene, enabling phenomena like reflections (fire a ray in the reflection direction), refraction (bend the ray according to Snell’s law), and shadows (fire a ray toward the light and check if anything blocks it). For rasterization, accessing this global spatial information requires the elaborate approximations discussed in Section 2.

3.2 Ray-Triangle Intersection: The Moller-Trumbore Algorithm

The innermost operation of a ray tracer is determining whether and where a ray intersects a triangle. The **Moller-Trumbore Algorithm** (1997) is the dominant method used in production ray tracers due to its efficiency - it avoids computing the triangle’s plane equation entirely.

A ray is defined parametrically as $r(t) = \vec{o} + t\vec{d}$, where $\vec{o}$ is the origin and $\vec{d}$ is the normalized direction. A triangle is defined by three vertices $\vec{p}_0, \vec{p}_1, \vec{p}_2$, with edge vectors $\vec{e}_1 = \vec{p}_1 - \vec{p}_0$ and $\vec{e}_2 = \vec{p}_2 - \vec{p}_0$.

Setting the ray equal to the barycentric parameterization of the triangle gives the linear system:

$$\vec{o} + t\vec{d} = \vec{p}_0 + u\vec{e}_1 + v\vec{e}_2$$

Rearranging and solving via Cramer's rule:

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{(\vec{d} \times \vec{e}_2) \cdot \vec{e}_1} \begin{bmatrix} (\vec{o} - \vec{p}_0) \cdot (\vec{e}_1 \times \vec{e}_2) \\ (\vec{d} \times \vec{e}_2) \cdot (\vec{o} - \vec{p}_0) \\ (\vec{o} - \vec{p}_0) \times \vec{e}_1 \cdot \vec{d} \end{bmatrix}$$

An intersection is valid if $t > 0$ (intersection is ahead of the ray origin), and $u \geq 0, v \geq 0, u + v \leq 1$ (intersection point is inside the triangle in barycentric coordinates). The algorithm requires 1 division, 27 multiplications, and 17 additions, and maps efficiently to GPU hardware.

3.3 Bounding Volume Hierarchies (BVH)

Evaluating the Moller-Trumbore test against every triangle in a scene containing millions of primitives would be computationally catastrophic. The standard acceleration structure for ray tracing is the **Bounding Volume Hierarchy (BVH)**, a tree structure where each internal node contains a bounding volume (typically an Axis-Aligned Bounding Box, or AABB) that tightly encloses all geometry in its subtree.

A ray traverses the BVH by testing against the bounding box of each node. If the ray misses the bounding box, the entire subtree is skipped. If it hits the bounding box, the algorithm recurses into the children. Leaf nodes contain the actual triangle data for exact intersection testing. This transforms a linear $O(N)$ search through triangles into an average-case $O(\log N)$ tree traversal.

3.3.1 The Two-Level BVH: BLAS and TLAS Modern ray tracing APIs (DirectX Raytracing, Vulkan Ray Tracing) organize scene acceleration structures into a two-level hierarchy:

- **Bottom-Level Acceleration Structures (BLAS):** Each BLAS contains the BVH for a single mesh in its local object space. BLASes can be reused - a single BLAS for a rock mesh can be instanced thousands of times in the scene without duplicating the triangle data.
- **Top-Level Acceleration Structure (TLAS):** The TLAS contains all instances in the scene, each referencing a BLAS plus a 4×3 transformation matrix (translation, rotation, scale). During ray traversal, when a ray hits

an instance bounding box in the TLAS, the ray is transformed into the BLAS's local coordinate space before descending into the BLAS tree.

This separation allows dynamic scenes to be handled efficiently: animated objects can rebuild or refit their BLASes every frame without requiring a full TLAS rebuild. The TLAS is rebuilt each frame as objects move, but at a cost proportional to the number of instances, not the number of triangles.

3.3.2 Surface Area Heuristic (SAH) for BVH Construction The quality of a BVH - and therefore the efficiency of ray traversal - depends critically on how the tree is constructed. The **Surface Area Heuristic (SAH)** provides a cost model for comparing candidate tree splits. The intuition is that the probability of a random ray hitting an AABB is proportional to the surface area of that AABB. For a proposed split dividing N triangles into two child sets A (with N_A triangles) and B (with N_B triangles):

$$C_{split} = C_T + \frac{SA(A)}{SA(parent)} \cdot N_A \cdot C_I + \frac{SA(B)}{SA(parent)} \cdot N_B \cdot C_I$$

Where C_T is the cost of a tree node traversal (bounding box test), C_I is the cost of a triangle intersection test, and $SA(\cdot)$ denotes the surface area of a bounding box. By evaluating SAH cost for many candidate split positions and choosing the minimum, BVH builders produce high-quality trees that minimize average ray traversal cost.

3.4 Hybrid Rasterization + Ray Tracing Pipelines

Despite the physical elegance of full ray tracing, its computational cost remains substantial - typically 5 to 20 times more expensive than an equivalent rasterized approximation for the same effect. Modern game engines therefore use a **hybrid pipeline** that strategically combines rasterization and ray tracing:

Rendering Effect	Rasterization Approach	Ray Tracing Approach	Notes
Primary Visibility	G-Buffer Pass	Primary ray cast	Rasterization far faster here
Hard Shadows	Shadow Maps (fast)	Shadow rays (accurate)	RT enables correct contact shadows
Soft Shadows	PCSS (approximate)	Stochastic shadow rays	RT gives physically correct penumbras

Rendering Effect	Rasterization Approach	Ray Tracing Approach	Notes
Reflections	SSR + Cubemaps (limited)	Reflection rays (exact)	RT shows off-screen geometry
Ambient Occlusion	SSAO/HBAO (screen-space only)	AO rays (full-scene)	RT eliminates screen-edge errors
Global Illumination	Baked lightmaps / SSGI	Diffuse GI rays	RT gives dynamic, accurate bounces

The rasterizer handles the primary visibility problem (determining what is visible from the camera), as it remains dramatically more efficient at this task. Hardware ray tracing then dispatches targeted ray queries to resolve the physically complex secondary effects.

3.5 Path Tracing: Unbiased Monte Carlo Integration

Path Tracing is the gold standard of light transport simulation, abandoning all approximations in favor of directly integrating the Rendering Equation via Monte Carlo methods. Rather than computing lighting effects in isolated passes, a path tracer shoots one or more rays per pixel, and each ray recursively bounces through the scene, physically simulating the complete multi-bounce light transport from light sources to the camera.

3.5.1 Monte Carlo Estimation The Monte Carlo estimate of the rendering equation integral is:

$$L_o(p, \omega_o) \approx L_e(p, \omega_o) + \frac{1}{N} \sum_{j=1}^N \frac{f_r(p, \omega_j, \omega_o) L_i(p, \omega_j) (n \cdot \omega_j)}{p(\omega_j)}$$

Where ω_j is the j -th sampled direction drawn from a probability density function $p(\omega_j)$. By the law of large numbers, as $N \rightarrow \infty$, the estimator converges to the true integral. The key challenge is that variance (and thus visible noise) decreases as $O(1/\sqrt{N})$, meaning doubling the sample count reduces noise by only $\sqrt{2} \approx 1.41\times$.

3.5.2 Importance Sampling and Multiple Importance Sampling (MIS)

Naive uniform sampling of directions over the hemisphere results in high variance, particularly for materials with narrow specular lobes or for scenes with small, intense light sources. **Importance Sampling** dramatically reduces variance

by sampling directions ω_j from a distribution $p(\omega_j)$ that is proportional to the integrand.

- **BRDF Importance Sampling:** Sample directions according to the shape of the BRDF lobe. For GGX specular materials, this means concentrating samples near the specular peak direction, proportional to $D(h)(n \cdot h)$.
- **Light Importance Sampling (Next Event Estimation):** At each surface interaction, explicitly sample a point on each light source and evaluate the shadow ray, directly estimating the direct illumination contribution.
- **Multiple Importance Sampling (MIS):** Combine BRDF and light sampling using the **Veach-Guibas balance heuristic** weights, reducing variance in both glossy and diffuse scenarios simultaneously:

$$w_k(\omega) = \frac{n_k p_k(\omega)}{\sum_j n_j p_j(\omega)}$$

3.5.3 Russian Roulette Path Termination Tracing a ray through every bounce until it reaches a light source or exits the scene is computationally intractable. **Russian Roulette** provides an unbiased stochastic termination strategy: at each bounce, a random number is drawn; if it exceeds a continuation probability q (typically the maximum reflectance channel of the surface), the path is terminated. To maintain mathematical unbiasedness, surviving paths have their radiance contribution amplified by $1/q$, preserving the expected value of the estimator.

3.5.4 Real-Time Path Tracing: Cyberpunk 2077 Overdrive Mode The implementation of full path tracing in *Cyberpunk 2077* Overdrive mode (2023) represents the most prominent real-time path tracing deployment to date. The pipeline fires 1-4 rays per pixel per frame for multiple effect categories (direct shadows, specular reflections, diffuse global illumination). The unfiltered output at these sample counts is extremely noisy and only becomes acceptable after aggressive spatiotemporal denoising. Even on flagship hardware (NVIDIA RTX 4090), Overdrive mode runs at approximately 30-60 FPS at 4K only because it relies entirely on DLSS 3.5 with ray reconstruction and Frame Generation to multiply the effective output frame rate from an internally rendered ~15 FPS. This vividly illustrates the computational demands of real-time path tracing.

3.6 Spatiotemporal Denoising: SVGF

Denoising is not optional in real-time ray tracing - it is an architectural necessity. With only 1-2 samples per pixel per frame, the Monte Carlo estimator has insufficient samples to converge, producing a raw output dominated by high-frequency stochastic noise. Denoising filters must reconstruct a clean, plausible image by exploiting spatial coherence (neighboring pixels share similar lighting) and temporal coherence (the scene changes slowly between frames).

3.6.1 Motion Vectors and Temporal Reprojection The cornerstone of spatiotemporal denoisers is **temporal reprojection**. During rendering, the engine computes a **motion vector** for each pixel - a 2D screen-space offset from the current pixel’s location to where that same surface point was located in the previous frame. This is computed by projecting the pixel’s world-space position through the previous frame’s view-projection matrix and subtracting the current pixel coordinates.

Using motion vectors, the denoiser can retrieve the lighting estimate from the previous frame for the same surface point (even if the camera moved), accumulating information across many frames. For a point in view for N frames, this effectively multiplies the sample count to N samples-per-pixel.

3.6.2 SVGF: Variance-Guided Filtering Spatiotemporal Variance-Guided Filtering (SVGF), introduced by Schied et al. (2017), formalizes this approach into a complete denoiser pipeline. Its temporal accumulation uses an exponential moving average:

$$C_{accum}^{(t)} = \alpha \cdot C_{current}^{(t)} + (1 - \alpha) \cdot C_{accum}^{(t-1)}$$

A small α (e.g., 0.05-0.20) retains most of the history, maximizing effective sample count. However, the denoiser simultaneously tracks the accumulated variance:

$$\sigma_{accum}^2 = \alpha \cdot \sigma_{current}^2 + (1 - \alpha) \cdot (\sigma_{accum}^2 + (\mu - \mu_{old})^2)$$

If the tracked variance spikes (indicating that the lighting environment has changed or the surface has been disoccluded), the blend factor α is automatically increased toward 1.0, rapidly discarding stale history and accepting the noisy current frame. Spatially, SVGF applies a series of **A-Trous wavelet filter** passes. Each pass doubles the sampling radius, efficiently implementing a large spatial filter without a correspondingly large convolution kernel. The key to preserving edges is the edge-stopping function that weights samples based on geometric similarity:

$$w(p, q) = \exp \left(-\frac{|z_p - z_q|}{\sigma_z |\nabla z_p| + \epsilon} - \frac{\arccos(n_p \cdot n_q)^2}{\sigma_n} - \frac{|l_p - l_q|^2}{\sigma_l \sigma^2} \right)$$

Samples at position q are down-weighted if they differ significantly in depth (z), surface normal (n), or luminance (l) from the center pixel p , preventing the filter from blending across geometric edges where separate illumination environments exist.

4. Next-Generation Geometry Pipelines and Virtualization

4.1 The Traditional Geometry Pipeline’s Scaling Problem

As game world detail and polygon counts have grown exponentially, the traditional mesh rendering pipeline has confronted fundamental bottlenecks at both the CPU and GPU level.

On the **CPU side**, the traditional pipeline requires the CPU to: 1. Traverse the scene hierarchy to identify visible objects (frustum culling). 2. Determine the appropriate LOD for each object based on camera distance. 3. Bind vertex/index buffers, constant buffers, and shader state for each object. 4. Issue a **Draw Call** through the API, which serializes state changes and transfers control metadata to the GPU.

At scene complexity levels common in modern open-world games (tens of thousands of unique objects), this CPU workload consumes 5-10 milliseconds per frame - an unacceptable fraction of a 16ms budget.

4.2 GPU-Driven Rendering and Indirect Draw Calls

GPU-driven rendering restructures the visibility culling pipeline to execute entirely on the GPU. Instead of the CPU issuing thousands of Draw Calls, it issues a small number of **Indirect Draw Calls** whose parameters are written by a compute shader running on the GPU. The visibility compute shader performs:

1. **Frustum Culling:** For each object’s bounding sphere, test whether it intersects the camera frustum. Discard invisible objects.
2. **Occlusion Culling using Hierarchical Z (HZB):** Build a mipmap pyramid of the depth buffer from the previous frame. Test each object’s bounding box against the appropriate HZB mip level. If the depth of the bounding box’s nearest point is greater than the recorded HZB depth, the object is likely occluded and is discarded.
3. **LOD Selection:** Compute the projected screen-space diameter of each object and select the appropriate LOD level.
4. **Command Buffer Population:** Write the draw parameters into a GPU buffer consumed by subsequent indirect draw calls.

This architecture eliminates CPU overhead for per-object culling entirely, allowing a single compute dispatch to process tens of thousands of objects in milliseconds.

4.3 Mesh Shaders: Replacing the Geometry Pipeline

Mesh Shaders (supported in DirectX 12 Ultimate and Vulkan with `VK_EXT_mesh_shader`) replace the entire IA/VS/GS pipeline with two compute-like stages:

- **Task Shader (Amplification Shader):** Runs once per work group of Meshlets. It can evaluate culling conditions (frustum, backface, occlusion) for each Meshlet and conditionally spawn a Mesh Shader workgroup or

discard the Meshlet entirely. This enables GPU-side per-Meshlet culling before any geometry enters the rasterizer.

- **Mesh Shader:** Runs per-Meshlet (typically 64-128 threads per workgroup). It computes vertex positions and primitive connectivity, outputting both vertex attributes and primitive index lists in compute-like shared memory. The GPU then proceeds to rasterize the emitted geometry as normal.

Meshlets are pre-computed during content processing: the 3D mesh is partitioned into compact clusters (approximately 64 vertices and 126 triangles per Meshlet) using spatial grouping algorithms, with the goal of maximizing vertex cache coherency within each Meshlet.

4.4 Virtualized Micro-Polygon Geometry: The Nanite Architecture

Nanite (Unreal Engine 5, 2021) represents the complete architectural reimagining of geometric detail management. Its foundational insight is that at sufficiently high polygon counts, no individual triangle should ever project larger than one pixel - if geometry is fine enough, the pixel shader’s texture resolution is the limiting detail budget, not the mesh.

4.4.1 Cluster Hierarchy as a Directed Acyclic Graph (DAG) During asset import, Nanite constructs a **hierarchical cluster DAG** of the mesh. The full-resolution mesh is partitioned into clusters of 128 triangles. Adjacent clusters are then merged and re-partitioned at half the resolution, producing parent clusters that approximate the original geometry less precisely. This process recurses, producing a tree of cluster groups from lowest (entire mesh as a few coarse clusters) to highest (original full-resolution clusters).

The critical property of this hierarchy is that at each level, a **parent cluster’s surface area error** relative to the full-resolution geometry is explicitly tracked. This error metric allows the runtime system to decide, for any given screen-space projection, which LOD level provides “good enough” approximation such that the error is sub-pixel (invisible to the viewer).

4.4.2 Runtime LOD Selection and Streaming At runtime, Nanite executes a compute shader that traverses the cluster DAG and evaluates each cluster’s projected screen-space error. Clusters whose error is larger than a threshold (roughly one pixel) are refined - their child clusters are selected for rendering. Only the clusters actually selected for the current frame’s view are streamed from storage (NVMe SSD) into VRAM, providing **virtual geometry** semantics: the full geometric detail is logically present, but only the view-relevant subset is physically resident in memory at any time.

4.4.3 The Visibility Buffer Paradigm Rendering billions of sub-pixel triangles through a traditional G-Buffer pass would be catastrophically expensive in bandwidth and memory. Nanite instead employs the **Visibility Buffer** approach.

Rather than writing full shading data (albedo, normals, roughness - possibly 48+ bytes per pixel) to a G-Buffer during the geometry pass, the rasterizer writes only a compact **64-bit integer** per pixel: the upper 32 bits encode the **InstanceID** (which object) and the lower 32 bits encode the **TriangleID** (which triangle within that object’s cluster).

A subsequent compute shader reads this integer, reconstructs the full barycentric coordinates of the visible triangle, fetches texture data for the material, and evaluates the full BRDF. This deferred evaluation means shading work is performed exactly once per visible pixel, regardless of overdraw.

4.4.4 Software Rasterization for Sub-Pixel Triangles When triangles are smaller than a pixel (common when Nanite renders highly detailed assets at moderate distances), the fixed-function hardware rasterizer becomes inefficient. For these sub-pixel clusters, Nanite bypasses the hardware rasterizer entirely and employs **software rasterization** - a compute shader that directly evaluates the edge equations for tiny triangles and atomically writes to the visibility buffer using 64-bit atomic compare-and-swap operations to handle visibility testing. This software path is highly optimized for the specific case of dense, small triangles, providing dramatically better throughput than the fixed-function hardware in this regime.

5. AI-Driven Reconstruction and Frame Generation

5.1 The Resolution-Performance Paradox

The relentless progression of display technology has created a fundamental tension in real-time rendering. Modern gaming monitors and televisions now standardly operate at **4K UHD resolution** ($3840 \times 2160 = 8,294,400$ pixels), with premium models targeting **120 Hz** or even **240 Hz** refresh rates. At the same time, rendering workloads have grown substantially due to hardware ray tracing, path tracing, and Nanite’s enormous polygon counts.

To render every one of 8.29 million pixels natively, with full physically-based shading, hardware ray tracing, and post-processing, at 120 frames per second, would require executing approximately one trillion shader operations per second. Even the most powerful consumer GPUs as of 2024 (NVIDIA RTX 4090, with ~82.6 TFLOPS of FP32 throughput) fall substantially short of this target when accounting for memory bandwidth limitations, algorithmic overhead, and the complexity of the rendering pipeline.

The resolution-performance paradox is thus not merely a transient hardware limitation but a fundamental scaling challenge: display resolution grows as $O(n^2)$ with linear dimension, while GPU performance growth has slowed considerably below historical rates since the end of Dennard scaling.

5.2 Anti-Aliasing and the Sampling Problem

Before discussing upscaling, it is important to understand the aliasing problem it also addresses. When rendering a 3D scene to a discrete pixel grid, **spatial aliasing** arises from undersampling: geometric edges that fall between pixel centers produce jagged “staircase” artifacts (**jaggies**). This is a fundamental consequence of the Nyquist-Shannon sampling theorem, which states that a signal must be sampled at more than twice its highest frequency to be represented without aliasing.

5.2.1 Multisample Anti-Aliasing (MSAA) MSAA evaluates coverage and depth at multiple sub-pixel sample positions per pixel (typically 2x, 4x, or 8x), but executes the pixel shader only once per pixel. If multiple samples within a pixel are covered by a triangle, the pixel shader output is written to each covered sample, and the final pixel color is the average of all sample values. MSAA effectively eliminates geometric aliasing without multiplying pixel shader cost. However, MSAA only addresses geometric edges; it provides no benefit for aliasing caused by specular highlights or high-frequency textures.

5.2.2 Temporal Anti-Aliasing (TAA) TAA amortizes super-sampling cost over time by accumulating sub-pixel samples across consecutive frames. Each frame, a sub-pixel **jitter** offset is applied to the camera projection matrix, using a low-discrepancy sequence (such as the Halton sequence) that ensures good coverage of the pixel area over multiple frames. This means that over 8 frames, each pixel effectively samples 8 different sub-pixel positions - equivalent to 8x MSAA at no additional per-frame cost.

The challenge is that between frames, the camera and objects move. TAA uses **motion vectors** to reproject the accumulated history buffer to align with the current frame before blending. To prevent **ghosting artifacts** (blurring of fast-moving objects), TAA employs **neighborhood clamping**: the colors of the 3x3 neighborhood around each pixel in the current frame are used to construct a **YCoCg color bounding box**, and the historical sample is clamped into this box before blending, preventing temporally invalid stale data from contributing to the final color.

5.3 Spatial Upscaling Techniques

Spatial upscaling analyzes only the current frame’s pixels to reconstruct a higher-resolution output. Algorithms range from simple bilinear interpolation to more sophisticated methods like **Lanczos resampling** (a windowed sinc function that preserves high-frequency detail better) and AMD’s **FidelityFX Super Resolution 1 (FSR 1)**, which uses a two-pass algorithm: EASU (Edge Adaptive Spatial Upsampling) detects edges using gradient analysis and adapts the filter kernel to avoid blurring across edges, followed by RCAS (Robust Contrast Adaptive Sharpening) to restore high-frequency texture detail.

Spatial upscaling is computationally inexpensive (typically under 1ms) but fundamentally limited: it can only work with information present in the current low-resolution frame. It cannot reconstruct detail that is sub-pixel or off-screen.

5.4 Temporal Upscaling: DLSS, FSR 2, and XeSS

Temporal upscaling combines the current frame with a history buffer of previous frames to reconstruct detail that was never rendered at the output resolution. It is fundamentally more powerful than spatial upscaling because temporal accumulation is equivalent to increasing the effective sample count per pixel.

5.4.1 NVIDIA DLSS (Deep Learning Super Sampling) DLSS reconceptualizes upscaling as a machine learning inference problem. DLSS 2.0 and later are **generalized**: a single trained neural network model handles all games and resolutions.

The DLSS inference network receives as input: * The current low-resolution aliased color frame (e.g., 1080p) * The exponentially averaged history buffer from previous frames (reprojected using motion vectors) * Per-pixel motion vectors (in the full output resolution) * Per-pixel depth values

The network outputs a high-resolution frame (e.g., 4K). It is a **Convolutional Neural Network (CNN)** with a U-Net-style encoder-decoder architecture, using attention mechanisms and residual connections. The model is trained offline on NVIDIA’s supercomputing cluster using 16K-resolution path-traced ground-truth images as training targets, using a perceptual loss function that evaluates sharpness and temporal stability as well as pixel-level accuracy.

Critically, the neural network inference is executed on dedicated **Tensor Core** hardware present in NVIDIA RTX GPUs. Tensor Cores perform **matrix-multiply-accumulate (MMA)** operations on 4×4 (or 8×16 in newer Tensor Core generations) matrices per clock cycle, in mixed-precision arithmetic (FP16/BF16 inputs, FP32 accumulation). This allows the complex convolutional inference to complete in approximately 1-2 milliseconds per frame.

DLSS Mode	Internal Resolution (% of Output)	Approx. Performance Multiplier
Ultra Quality	77%	~1.5-1.7x
Quality	67%	~2.0-2.3x
Balanced	58%	~2.5-3.0x
Performance	50%	~3.0-4.0x
Ultra Performance	33%	~5.0-6.0x

DLSS 3.5 (2023) added **Ray Reconstruction**, which replaces the traditional denoiser for ray-traced effects. Rather than a manually-designed edge-stopping

filter, a neural network trained on ground-truth path-traced data directly reconstructs clean ray-traced reflections, shadows, and ambient occlusion from the noisy 1-2 spp ray tracer output, producing substantially better results than algorithmic denoisers.

5.4.2 AMD FidelityFX Super Resolution 2 (FSR 2) AMD’s **FSR 2** achieves temporal upscaling without a dedicated neural network or specialized hardware, making it compatible with a broad range of GPU vendors including NVIDIA and Intel. FSR 2 uses manually-engineered algorithms to replicate the core functions of a temporal upscaler:

- **Motion Vector Analysis:** Reconstruct sub-pixel geometry positions using the engine-provided motion vectors.
- **Reactive Mask:** The engine provides an optional per-pixel “reactivity” mask indicating which pixels contain transparent or rapidly-changing content (particles, hair, reflections). Pixels with high reactivity use a shorter history accumulation length to avoid ghosting.
- **Temporal Filter and History Reconstruction:** The algorithm blends current and historical samples using Lanczos filtering for anti-aliasing, combined with luminance-based history rejection to prevent ghosting.

FSR 2 produces visual quality competitive with DLSS 2 at comparable upscaling ratios, though it can exhibit more instability on highly specular or complex translucent surfaces where the neural network’s learned priors give DLSS an advantage.

5.4.3 Intel XeSS (Xe Super Sampling) Intel **XeSS** offers a hybrid approach: on Intel Arc GPUs with dedicated DP4a (Dot Product 4 8-bit with 32-bit accumulation) hardware, it runs an optimized neural network inference path. On GPUs from other vendors lacking this hardware, it falls back to an algorithmic temporal upscaling path similar in design to FSR 2. The XeSS neural network was trained using technology similar to DLSS, targeting perceptual quality at reasonable computational cost.

5.5 Frame Generation: Optical Flow Synthesis

Even with upscaling quadrupling rendering performance, some workloads (particularly CPU-bound scenarios with complex physics simulations) cannot achieve target frame rates. **Frame Generation** (NVIDIA DLSS 3 Frame Generation, AMD FSR 3 Frame Generation) addresses this by synthesizing entirely new, intermediate frames between rendered frames, doubling the displayed frame rate without increasing rendering frequency.

5.5.1 Optical Flow Estimation The core algorithm estimates **optical flow** - a per-pixel 2D displacement vector field describing how each pixel moves between two consecutive frames. Unlike geometric motion vectors (which describe how

rendered 3D scene geometry moved between frames), optical flow is estimated purely from the pixel data of two rendered frames and is therefore sensitive to all pixel-level changes: lighting variations, specular highlight movement, particle systems, and reflections.

The **Optical Flow Accelerator (OFA)** is dedicated hardware on NVIDIA GPUs that accelerates this computation using a multi-scale block-matching algorithm. Block-matching divides the frame into small blocks (e.g., 8x8 pixels) and finds the best matching block in the adjacent frame within a search radius, minimizing a sum-of-absolute-differences (SAD) cost:

$$\text{Flow}(x, y) = \arg \min_{\Delta x, \Delta y} \sum_{i, j \in \text{block}} |I_t(x + i, y + j) - I_{t-1}(x + i + \Delta x, y + j + \Delta y)|$$

This is computed hierarchically: a coarse optical flow estimate at low resolution is refined at progressively higher resolutions.

5.5.2 Frame Interpolation Using the optical flow field, a neural network synthesizes the interpolated frame. The network: 1. Warps Frame $N - 1$ forward in time using the optical flow, and warps Frame N backward in time. 2. Blends the two warped frames, resolving conflicts and artifacts at disocclusion boundaries (regions visible in one frame but not the other) and at the borders of moving objects. 3. Uses the engine’s geometric motion vectors to distinguish separately-moving objects, handling occlusion boundaries more accurately than optical flow alone.

The output is an intermediate synthetic frame at a temporal midpoint between Frame $N - 1$ and Frame N , visually plausible and temporally consistent with its neighbors.

5.5.3 Latency Implications and NVIDIA Reflex Frame Generation introduces a fundamental latency trade-off: because the synthesized frame is derived from two rendered frames, the most recently rendered frame must be held in a buffer while the intermediate frame is synthesized and inserted before it in the display queue. This adds a fixed latency of approximately one frame period (8.33ms at 120 Hz) to the time between the player’s input and the display of the corresponding rendered frame.

To counteract this additional latency, Frame Generation is mandatorily coupled with **NVIDIA Reflex** (or AMD’s equivalent Anti-Lag). Reflex instruments the CPU game thread and render thread to minimize the **render queue depth** - the number of CPU-submitted but GPU-unconsumed frames buffered in the driver. Traditional multi-frame buffering introduced 50-100ms of latency between input polling and rendering. Reflex reduces this to near-zero by synchronizing CPU and GPU progress, sampling input controller data at the last possible moment before submission. The net result is that the combined DLSS 3 + Reflex pipeline

often achieves lower total system latency than native rendering without Reflex, despite the addition of Frame Generation’s insertion delay.

6. Comparative Analysis and Future Directions

6.1 Rasterization vs. Ray Tracing: A Quantitative Comparison

The following table summarizes the key technical trade-offs between traditional rasterization and hardware-accelerated ray tracing for primary rendering effects:

Criterion	Rasterization	Ray Tracing
Algorithmic Complexity	$O(T \cdot P)$ (triangles x pixels covered)	$O(R \cdot \log T)$ (rays x BVH depth)
Physical Accuracy	Approximate (numerous cheats)	Exact (physically correct light transport)
Off-Screen Information	Unavailable (screen-space only)	Full scene available
Dynamic Scene Handling	Excellent (G-buffer rebuilt each frame)	TLAS/BLAS must be rebuilt/refitted
Performance Cost (Reflections)	Low (SSR: ~0.5ms)	High (RT reflections: ~3-8ms)
Performance Cost (Shadows)	Low-Medium (CSM: ~1-3ms)	Medium (RT shadows: ~2-5ms)
Visual Quality (Reflections)	Limited to screen-space, no off-screen	Full-scene, physically correct
Hardware Requirement	All DX11+ GPUs	DX12 Ultimate, RDNA2+, Ampere+

6.2 The Path to Full Real-Time Path Tracing

The trajectory of modern real-time rendering points toward full path tracing becoming the standard rendering algorithm within the next 5-10 years. The enabling factors are:

1. **AI Denoising:** Neural denoisers (like DLSS Ray Reconstruction) can reconstruct photorealistic images from 1-4 spp, the sample count achievable in real-time budgets.
2. **AI Upscaling:** DLSS and FSR allow internal render resolutions far below native output, multiplying effective path tracing performance by 4-6x.
3. **Frame Generation:** Further doubles effective frame rate, decoupling visual smoothness from render frequency.
4. **Hardware Advances:** Each GPU generation improves RT Core/Ray Accelerator throughput by 1.5-2x (NVIDIA Ada Lovelace RT Cores deliver 3x the ray-triangle throughput of Ampere).

The convergence of these technologies suggests that the incremental hybrid pipeline is a transitional stage. Full path tracing, with AI-driven reconstruction replacing real-time physical convergence, is the logical endpoint of the current technological trajectory.

Research carried out by Swastik Pandey

References

1. Kajiya, J. T. (1986). The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4), 143-150.
2. Williams, L. (1978). Casting curved shadows on curved surfaces. *ACM SIGGRAPH Computer Graphics*, 12(3), 270-274.
3. Cook, R. L., & Torrance, K. E. (1982). A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1), 7-24.
4. Moller, T., & Trumbore, B. (1997). Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1), 21-28.
5. Schied, C., et al. (2017). Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination. *Proceedings of High Performance Graphics*, Article 2.
6. Karis, B. (2013). Real shading in Unreal Engine 4. *SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice*.
7. Burley, B. (2012). Physically-based shading at Disney. *SIGGRAPH 2012 Course: Practical Physically Based Shading in Film and Game Production*.
8. Graham, M., et al. (2021). Nanite: A deep dive. *Unreal Engine 5 Technical Documentation*, Epic Games.
9. Veach, E., & Guibas, L. J. (1995). Optimally combining sampling techniques for Monte Carlo rendering. *Proceedings of SIGGRAPH 1995*, 419-428.
10. NVIDIA Corporation. (2023). DLSS 3.5 technical overview: Ray reconstruction. *NVIDIA Developer Documentation*.
11. Drobot, M. (2017). Hybrid ray traced shadows. *GPU Pro 7: Advanced Rendering Techniques*.
12. Ubisoft Montreal. (2015). GPU-driven rendering pipelines. *SIGGRAPH 2015 Advances in Real-Time Rendering Course*.
13. Wald, I. (2007). On fast construction of SAH-based bounding volume hierarchies. *IEEE Symposium on Interactive Ray Tracing*, 33-40.
14. Persson, E. (2012). Practical clustered shading. *SIGGRAPH 2012 Advances in Real-Time Rendering Course*.
15. Hasselgren, J., & Akenine-Moller, T. (2006). PCU: The programmable culling unit. *ACM Transactions on Graphics*, 25(3).